

完成人生第一个 Vibe Coding Web Application

几个小时->从零到上线

初始目标与思路

作为一个没有深厚前端背景的开发者的挑战者，我今天的挑战很明确：在不手写大量代码的前提下，使用 Vibe Coding 完成一个 Self-contained (自包含) 的单页 Web Application。

我的核心思路是“意图驱动”。我不关心具体的 CSS 属性怎么写，我只关心最终的交互逻辑和视觉效果。因此，我决定采用“需求文档 + 视觉草图”的双重输入模式，让大模型（智谱 GLM）成为我的“全栈外包团队”，而我只需要端着茶杯，静静坐在指挥官的位置上进行验收。

任务拆解与迭代过程

整个 workflow 依托于 Windows + Visual Studio Code + Cline 插件。

1. 静态定义阶段：我没有急于生成代码，而是先准备了一份详细的

showcase.md 和一个 **layout.jpeg**。这一步相当于完成了 **Junior Business Analyst** 和 **Junior UI Designer** 的工作。

2.Agent 执行阶段：将文件投喂给 Cline（调用智谱 GLM）。Cline 会根据指令直接创建文件、编写代码。

3.反馈闭环：在 Edge 浏览器中预览页面，哪里样式不对或逻辑缺失，我就把问题反馈给 AI，让它进行增量修改。

这种模式下，开发不再是线性编码，而是“生成-预览-修正”的快速循环。

大模型的表现评估

这次使用的智谱 GLM 表现非常惊艳，特别是在理解多模态输入方面：

1.视觉对齐能力：得益于我提供的 JPEG Mockup，GLM 生成的布局还原度很高，没有出现严重的结构错乱。

2.逻辑连贯性：对于 Self-contained 单页应用的理解很到位，没有过度引入不必要的外部依赖。

3.工程化收尾：最让我惊喜的是，在任务结束时，它能按照我的要求，自动汇总所有文件，生成了一份结构清晰的 **README.md**，详细阐述了设计理念和目录结构。这一点大大超出了我对单纯“代码补全”的预期。

问题与修复策略

当然, Vibe Coding 并非没有坑。我在中期遇到了典型的 Context Window (上下文窗口) 超载问题。

随着迭代次数增加, 对话历史越来越长, 导致 API 突然停止响应, 模型无法处理新的指令。作为新手, 我一开始忽略了 Token 的限制。

修复策略: Task ID 续接法

我没有选择重头再来, 而是利用了 Cline 的特性。我找到了上一个崩溃任务的 Task ID, 然后开启一个新的对话, 指令新任务: “请读取该<Task ID>的上下文, 并接着完成未竟的工作。” 通过这种方式, 我成功恢复了工作状态, 避免了上一个任务失败的重启成本。这也让我意识到, 管理上下文 (**Context Management**) 是 Vibe Coding 的核心技能。

最终总结

这次实践让我总结出一套**初级的** Vibe Coding 方法论:

- 1.先设计后编码: Mockup 和 Markdown 是必须的“蓝图”。
- 2.工具链整合: VSCode + Cline + 国产大模型 (GLM) 的组合已经非常成熟, 足以应对原型开发。
- 3.上下文工程: 不要把所有东西都堆在一个对话里。学会利用 Task ID 或外部文

档来“接力”，是完成复杂项目的必经之路。

作品的真实价值

这个 Web App 本身的代码或许不是最优美的，但它证明了 Vibe Coding 的真正价值在于“将想法转化为生产力的速度”。

过去，一个不懂 Web App 开发的人想做一个 Demo，可能需要学几周的前端知识；现在，只需要一下午。这种能力极大地拉平了技术鸿沟。对我而言，这不仅是一个应用的诞生，更是我从“Coder”向“Architect & Prompt Engineer”转型的一次成功验证。我已经录制了视频准备分享这次经验，因为我相信，这就是未来的工作方式。

同时，这次 Vibe Coding 生成的提升工作效率的 Web 小工具，也就被发布到 GitHub 或者通过文件共享的形式发给同事们，方便我们日常工作。